CECS 229 – Assignment 5

Cryptography Functions

Given the file, 'pa5.py', complete the code for the following functions:

1. `shift_encrypt(text, k)` – given plaintext and a key, this function returns the ciphertext encrypted using a shift cipher with key k.
    a. text – a string of the plaintext. This string will only consist of upper and lower-case letters and spaces, no other characters (ex. punctuation or numbers) will be included.
    b. k – an integer shift value.
    c. The returned value will be the encrypted string.
    d. Convert the text to all upper-case letters. Loop through each character in the text string, if it is a space, add it to the encrypted string, otherwise use the shift value to encrypt the character and then concatenate it onto the encrypted string. Return the encrypted string, which should only consist of upper-case letters and spaces.
2. `shift_decrypt(text, k)` – given ciphertext and a key, this function returns the plaintext decrypted using a shift cipher with key k. Note: this function only returns readable plaintext if the same key as the encryption process is used.
    a. text – a string of the ciphertext. This string will only consist of upper-case letters and spaces, no other characters will be included.
    b. k – an integer shift value.
    c. The returned value will be the decrypted string.
    d. Loop through each character in the text string, if it is a space, add it to the decrypted string, otherwise use the shift value to decrypt the character and then concatenate it onto the decrypted string. Return the decrypted string, which should only consist of upper-case letters and spaces.
3. `rsa_encrypt(text, n, e)` – this function uses RSA to encrypt the text using key (n, e).
    a. text – a string of the plaintext.
    b. n – is an integer that is the modulus for the RSA encryption function.
    c. e – is an integer that is the exponent for the RSA encryption function.
    d. The returned value is the encrypted number string.
    e. Remove any spaces from the text and convert it to upper case. Then calculate the blocksize using n and use that value to determine if the text needs to be padded with 'X's to fulfill the final block. Loop through the text and convert each character to its corresponding number (00-25). Break up the number string into individual blocks, then for each of the blocks, apply the RSA encryption function to it and add the result to the final encrypted number string (note: when calculating this result, if the value is smaller than the block length, then the front needs to be padded with '0's). Return the resulting encrypted number string.
4. `rsa_decrypt(text, p, q, e)` – this function uses RSA to decrypt the encrypted number string using key (p*q, e).
    a. text – a string of the ciphertext.
    b. p – the first prime number for the integer product that is the modulus in the RSA key.

c. q - the second prime number for the integer product that is the modulus in the RSA key.
d. e – is an integer that is the exponent in the RSA key.
e. The returned value is the decrypted text.
f. Calculate the blocksize using p and q and use that value to break up the encrypted number string into individual blocks. Then, for each of the blocks, apply the RSA decryption function to it and add the result to the decrypted number string (note: when calculating this result, if the value is smaller than the block length, then the front needs to be padded with '0's). Finally, loop through the number string and translate each pair of digits into a string of letters. Return the decrypted message.

Provided in the 'pa5.py' file are three utility functions for you to use in your code:

1. num_to_char(num) – this function passes in a number (string or int) and translates it into the corresponding capital letter. Ex: 00 = A, 01 = B, …, 25 = Z.
2. char_to_num(c) – this function passes in a letter and translates it into the corresponding number as a string. Ex: A = 00, B = 01, …, Z = 25.
3. blocksize(n) – this function uses the "25" rule to calculate the blocksize for the RSA encryption/decryption functions using the value n from the RSA key.
4. Your code will also need to use the mod_inv(a, m) and bezout_coeffs(a, b) functions that you wrote in Assignments 3 and 4, so be sure to copy those over.

Example Input/Output:

```
shift_encrypt("pirate attack", 10)
ZSBKDO KDDKMU
shift_encrypt("Zebra", 4)
DIFVE
shift_decrypt("ZSBKDO KDDKMU", 10)
PIRATE ATTACK
shift_decrypt("DIFVE", 4)
ZEBRA
rsa_encrypt("pirate attack", 8633, 13)
165113303456014962683605
rsa_encrypt("Zebra", 7597, 17)
644003696629
rsa_decrypt("165113303456014962683605", 97, 89, 13)
PIRATEATTACK
rsa_decrypt("644003696629", 107, 71, 17)
ZEBRAX
```

Notes:

1. Please place your name in the comment block at the top of the provided file.
2. Fill in the necessary code to complete the functions. Write your code where specified (ie. the TODOs). Do not modify the file name, function names, or parameters otherwise the program and tests will not work. Return the results in the format specified (ie. correct types and container types), otherwise the program and tests may fail.
3. Your solutions may not use any libraries or functions that perform the equivalent task.

4. Test your code using the provided ‘main.py’ file. You can write additional tests in the ‘your_tester.py’ file to further check your solutions. Note: just because the main tests pass does not guarantee that it will pass the tests on Gradescope.
5. When you are finished, submit your ‘pa5.py’ file to Gradescope on Canvas for further testing and grading. It should allow you to make repeated submissions.